/resume

Resume the agent from where it was halted.

Overview

`/resume` continues an agent run that was halted mid-task by the user (for example, by pressing Esc to cancel the in-flight response). The command injects a synthetic user message – "You were previously interrupted by the user. Continue the task from where you left off." – into the current session and spawns the agent loop again with the same session id, agent info, model and thinking settings, so the model picks up the task where the transcript left off. Halt state is tracked by the TUI: when a response finishes with `FinishReason::Cancelled`, the app sets `agent_halted = true` and the status bar shows "halted – /resume to continue". A normal completion clears the halt flag, and any new response start clears it as well.

Syntax

```
/resume
/resume help
```

Options / Subcommands

Form	Description
`/resume`	Continue a halted agent from where it was interrupted by the user
`/resume help`	Show per-command help (also accepted: `--help`, `-h`)

There are no further options or flags. The command takes no arguments besides the help forms.

Examples

Resume a halted run (status bar reads "halted – /resume to continue"):

```
/resume
```

Show the built-in help table:

```
/resume help
```

Typical sequence – cancel a long-running edit with Esc, then resume:

```
<Esc>           # cancels the response; agent_halted = true, status: halted
/resume         # agent continues the task from where it left off
```

Output

- When not halted: status line "Nothing to resume – agent was not halted" and a warning log entry ("Nothing to resume"); nothing is sent to the model.
- When there is no session: status line "No active session".

- When resumed: a new user message is appended to the transcript with the continuation instruction, the status shows “processing”, a log entry “Resuming halted agent” is written, and the agent’s response streams in as usual. The halt flag is cleared immediately.
- `/resume help` prints a markdown table of the two forms and sets the status to “resume: help”.

Related

- `/autopilot` – autonomous operation that can be combined with resume flows
- `/loop` – goal-driven loop programming with its own interrupt/stop guards
- `/undo` – remove the last user/assistant turn pair instead of continuing it